

Documento de regras para a ferramenta de anotação, treinamento e pipelines assistidos

Diretriz central do produto

A melhor direção para essa ferramenta é **não tentar substituir o CVAT**, e sim usá-lo como motor de anotação, revisão, jobs, tracking, auto-annotation e controle de qualidade, enquanto a sua interface funciona como uma camada de orquestração mais amigável para o usuário, com foco em revisão rápida, métricas, rastreabilidade, treinamento e composição de pipelines. Essa escolha é coerente com o que o próprio CVAT já expõe por SDK, CLI, REST API, webhooks, Track Mode, Ground Truth jobs e Auto-annotation API. ¹

Na prática, isso significa separar claramente três responsabilidades. O **CVAT** fica como fonte operacional de mídia, labels, shapes, tracks, revisão e QA; o **motor de treino e inferência** fica com Ultralytics e seus modelos de detecção, classificação, segmentação e tracking; e a **camada de produto** passa a gerenciar filas de revisão, configuração de treinos, painéis de métricas, pipelines encadeados, geração de datasets derivados e histórico auditável. Essa divisão reduz retrabalho e aproveita recursos que já existem de forma estável nos componentes oficiais. ²

A resposta curta para a sua pergunta é: **sim, dá para encaixar métricas em tempo real, histórico de evolução, controle de parâmetros e pipelines encadeados na interface e na máquina de estados**, mas a máquina atual precisa deixar de ser “uma única máquina para tudo” e passar a ser um conjunto de máquinas ortogonais: uma para o ciclo do projeto, uma para jobs assíncronos e uma para o estado de cada anotação individual. Essa separação combina melhor com a forma como o CVAT já modela estágio e estado de jobs, e também com a forma como treinamento, exportação e inferência realmente se comportam no backend. ³

Regras da máquina de estados e da experiência do usuário

A máquina principal deve representar o **ciclo do projeto**, não o detalhe interno de cada execução. O CVAT já distingue “stage” e “state” de jobs — `annotation`, `validation`, `acceptance` de um lado, e `new`, `in progress`, `rejected`, `completed` do outro. Isso é um bom sinal de que a sua ferramenta também deve separar fluxo de trabalho de progresso operacional. ⁴

A recomendação é adotar três máquinas coordenadas.

A primeira é a **máquina do projeto**: `Rascunho` → `Configurado` → `LoteSelecionado` → `Anotando/Pré-anotando` → `Revisando` → `QA` → `Release de Dataset` → `Treinando` → `Avaliando` → `Aprovado` → `Publicado`. O ponto importante aqui é que “treinando” e “avaliando” deixam de ser apenas setas e viram estados explícitos do produto. Isso permite abrir espaço na interface para monitoramento ao vivo, comparação histórica, retry controlado e aprovação formal antes de publicar um modelo. Essa estrutura também conversa bem com as features do CVAT para GT job, revisão e quality analytics. ⁵

A segunda é a **máquina do job assíncrono**: `na fila` → `executando` → `pausado` → `concluído` → `falhou` → `cancelado`. Essa máquina serve para treino, inferência, exportação, geração de crops, segmentação por prompt, avaliação e qualquer pipeline composto. Ela é necessária porque o usuário pode estar revisando um lote enquanto um treino anterior continua rodando em segundo plano. O próprio MLflow e o Ultralytics foram desenhados para runs, callbacks, métricas por época e artefatos associados a execuções específicas, o que combina com esse modelo de job assíncrono. ⁶

A terceira é a **máquina do item anotado**, que pode ser algo como `proposto` → `aceito` → `corrigindo` → `rejeitado` → `incerto` → `escalado`. Essa máquina é a base do modo de revisão rápida com teclado. O ideal é que a interface tenha um “modo decisão” explícito, separado do “modo edição”, para não conflitar com navegação em vídeo e ajuste fino de geometria. Como o CVAT já trabalha com Track Mode, keyframes, interpolação automática entre keyframes e marcação de `Outside`, faz sentido que sua camada de UX priorize decisões de revisão e só abra o editor completo quando a correção exigir geometria mais complexa. ⁷

Para vídeo, a regra mais importante é esta: **interpolação não deve ser tratada como um evento mágico de backend, mas como um recurso visível no fluxo de revisão**. O CVAT já interpola tracks entre keyframes e permite finalizar o track com `Outside`; a sua interface deve expor isso como um fluxo amigável: “aceitar segmento”, “corrigir keyframe”, “propagar até o próximo keyframe”, “aplicar classe ao track inteiro”, “dividir track aqui” e “encerrar track”. Isso é mais natural para o usuário do que mostrar apenas um canvas com objetos interpolados sem contexto. ⁷

A partir daí, a navegação por teclado que você imaginou faz muito sentido. Minha recomendação é institucionalizar isso como requisito de produto: `→` aceita e avança, `←` rejeita e avança, `↑` abre correção rápida de classe, `↓` marca como incerto, `1..9` troca para classes frequentes e confirma, `U` desfaz a última decisão, `Tab` muda de objeto, `Shift + ←/→` muda de frame. Isso não vem pronto nos documentos oficiais como um layout obrigatório, mas ele se encaixa bem no fato de o CVAT já suportar atalhos e edição orientada por objeto. ⁸

Regras para treinamento, monitoramento em tempo real e histórico de evolução

A sua ideia de manter um espaço específico para métricas de treinamento é extremamente forte e deve virar um **módulo de primeira classe da aplicação**, não apenas uma aba secundária. O motivo técnico é simples: o Ultralytics já salva métricas ao longo do treino, expõe `results.csv`, possui métodos para ler esse CSV, plotar métricas e conectar callbacks no ciclo de treino; o MLflow, por sua vez, foi feito exatamente para registrar parâmetros, métricas, artefatos e runs, inclusive com API REST para alimentar uma interface própria. ⁹

Para acompanhamento em tempo real, a regra ideal é que cada `TrainingRun` publique dois fluxos simultâneos. O primeiro é o **fluxo de métricas de modelo por época**, com loss, mAP, precision, recall, learning rate e quaisquer métricas personalizadas. O segundo é o **fluxo de métricas de sistema**, com CPU, GPU, memória, disco e rede. O MLflow suporta logging de métricas de sistema durante a execução de uma run, e o Ultralytics já integra logging de parâmetros, métricas e artefatos em MLflow por callbacks. Isso permite mostrar um painel vivo que responda a perguntas diferentes: “o modelo está melhorando?” e “a infraestrutura está saturada?”. ¹⁰

Esse módulo deve guardar, no mínimo, os seguintes grupos de dados por treino: identificador da run, dataset release usada, número de imagens, número de objetos, número de tracks, classes presentes,

hiperparâmetros, métricas por época, melhor checkpoint, métricas finais, tempo total, consumo de recursos e artefatos associados. O MLflow já provê tracking de experiments e runs, logging via API/REST, dataset tracking e model registry com vínculo entre versão do modelo e run de origem; por isso, a modelagem recomendada é aproveitar MLflow como back-end de experimento e a sua aplicação como front-end de análise. ¹¹

Para **gráficos em tempo real**, a recomendação de produto é ter quatro visões padrão. A primeira é “curvas por época”, com `train loss`, `val loss`, `mAP50-95`, `precision` e `recall`. A segunda é “saúde da máquina”, com GPU/CPU/memória. A terceira é “qualidade da validação”, com matriz de confusão, métricas por classe e, se possível, métricas por imagem. A quarta é “qualidade operacional”, com velocidade de treino, imagens por segundo e tempo estimado restante. O Ultralytics `val()` já expõe `mAP50`, `mAP75`, `mAP50-95`, métricas por classe e até métricas por imagem; o `BaseTrainer` possui leitura de `results.csv` e plotting de métricas. ¹²

Para **histórico de evolução**, a recomendação é que o eixo principal do gráfico não seja apenas “tempo”, mas também “tamanho e qualidade do dataset”. Em outras palavras, seu painel histórico deve conseguir plotar:

“release do dataset” → “melhor métrica do modelo”,

“objetos aceitos acumulados” → “mAP50-95”,

“tracks revisados acumulados” → “recall por classe”,

“tempo médio de revisão” → “ganho incremental do modelo”.

Esse tipo de correlação só fica confiável se cada treino estiver ligado a uma versão exata do dataset. O MLflow Dataset Tracking existe justamente para rastrear datasets usados em treino, validação e avaliação, com linhagem e versionamento. ¹³

Aqui existe uma regra de ouro: **não comparar curvas históricas sem um conjunto de validação fixo**. O CVAT suporta Ground Truth job em uma task, e o Ultralytics `val()` permite validar sempre contra um split definido, inclusive acessando métricas por categoria e por imagem. Portanto, a política correta é manter um holdout estável para comparação histórica, ou então marcar explicitamente quando uma curva estiver comparando releases sobre validações diferentes. Sem isso, a sensação de “melhora do modelo” pode ser ilusória. ⁵

Também vale reservar espaço para **métricas customizadas de domínio**. O Ultralytics permite customizar o trainer e logar métricas adicionais, como F1 médio e F1 por classe, mudando o trainer padrão ou usando callbacks no ciclo de validação. Isso é importante porque, em muitos projetos, a métrica que realmente importa não é apenas o mAP global, mas recall de uma classe crítica, taxa de false positive em determinado cenário, ou estabilidade de track em vídeo. ¹⁴

Se vocês quiserem evoluir para tuning mais sistemático, o MLflow também acomoda muito bem a ideia de um “estudo” com dezenas de tentativas. A documentação oficial mostra o uso de parent run e child runs para acompanhar ensaios de hyperparameter tuning e comparar os trials no UI. Isso casa perfeitamente com uma futura tela de “comparação de treinos”, onde o usuário filtra runs por dataset, backbone, batch, imagem, dispositivo e melhor checkpoint. ¹⁵

Regras para parâmetros de treinamento e governança de recursos

Faz muito sentido expor parâmetros de treino na interface, mas isso precisa ser feito com **camadas de complexidade**. O Ultralytics já aceita, entre outros, `epochs`, `batch`, `imgsz`, `device` e demais

ajustes via argumentos de treino e validação. A interface deve espelhar isso, mas sem jogar todos os parâmetros avançados no rosto do usuário desde o início. ¹⁶

A recomendação de UX é dividir a configuração em três grupos. O primeiro grupo é **Essencial**, para quem só precisa rodar um treino consistente: modelo base, dataset release, epochs, batch, tamanho da imagem e device. O segundo é **Performance**, com workers, cache, AMP, pin memory, persistent workers e número de GPUs. O terceiro é **Avançado**, com otimizador, learning rate, weight decay, scheduler, warmup, congelamento de backbone, gradient clipping, early stopping, fechamento de mosaic, thresholds de validação e métricas customizadas. Essa hierarquia bate com o fato de que o Ultralytics possui treino configurável por argumentos e também permite customização via trainer próprio. ¹⁷

Para batch e workers, a interface deve orientar sem tentar prometer um “valor mágico”. A documentação do PyTorch recomenda começar com batches moderados, usar `num_workers > 0` quando o carregamento de dados e transformações forem caros, iniciar em algo como 2–4 workers por GPU e crescer até o throughput estabilizar; ela também alerta que workers demais desperdiçam memória e podem estourar `/dev/shm`. Em paralelo, a documentação do próprio Ultralytics reforça que batch, learning rate, optimizer e composição do dataset impactam bastante a velocidade e a acurácia do treino. ¹⁸

Por isso, eu colocaria regras de produto muito claras. A interface pode **sugerir presets** como “econômico”, “equilibrado” e “agressivo”, mas sempre mostrar o que cada preset está configurando. Em modo manual, ela deve exibir avisos do tipo “batch alto aumenta uso de memória”, “workers altos podem aumentar throughput, mas também consumo de RAM e `/dev/shm`”, e “pin memory é recomendável quando houver acelerador”. Essas mensagens não são apenas UX; elas refletem recomendações explícitas da documentação do PyTorch. ¹⁹

Sobre “limitar o uso de recursos de máquina”, a resposta é: **sim, mas isso deve ser tratado como capacidade de infraestrutura, não apenas do script de treino**. O Ultralytics permite selecionar `device`, e o PyTorch permite otimizar carregamento e uso de memória, mas limites duros de CPU e memória são muito mais confiáveis quando o treino roda em contêineres ou pods com limites declarados. O Docker documenta explicitamente controles de CPU e memória para contêineres, e o Kubernetes documenta `requests` e `limits` para CPU e memória em containers e pods. ²⁰

Isso leva a uma regra arquitetural importante: **“controle de batch/workers” é ajuste de performance; “limite de máquina” é política de execução**. Logo, a interface deve ter dois níveis distintos. Um formulário configura o trainer. Outro define a política de execução do job: CPU máxima, memória máxima, GPU(s) permitidas, prioridade, fila, timeout, reexecução e isolamento. Se treinarem em Docker ou Kubernetes, esses limites podem ser realmente aplicados; se treinarem em um processo local fora de container, a interface deve deixar claro que o controle será apenas parcial. ²¹

Também recomendo uma regra operacional adicional: **todo treino deve registrar o ambiente de execução**. Isso inclui versão do Ultralytics, versão do PyTorch, device, imagem de contêiner, hostname/worker, número de CPUs, memória disponível e se havia ou não acelerador ativo. O racional é reproduzibilidade. O valor dessa regra fica ainda maior quando o modelo começa a ser comparado historicamente. MLflow, system metrics e model registry foram desenhados exatamente para esse tipo de rastreamento. ²²

Regras para pipelines encadeados e geração de datasets derivados

A sua ideia de encadear detecção, classificação e segmentação é boa, mas eu não a colocaria como uma sequência fixa “sempre nesta ordem”. Eu trataria isso como um **grafo de pipelines**. O motivo é que o encadeamento pode variar conforme o caso de uso: algumas tarefas pedem apenas detecção; outras pedem detecção seguida de classificação; outras pedem detecção seguida de segmentação; em vídeo, pode existir tracking entre a detecção e a geração de crops. O ecossistema Ultralytics suporta detecção, classificação, segmentação e tracking, e o SAM 2 aceita prompts por clique, caixa ou máscara em imagens e vídeos, o que o torna especialmente útil para o passo “caixa → máscara”. ²³

O encadeamento mais natural para o seu caso é este: **detecção → recorte → classificação → máscara opcional → revisão → release**. O primeiro passo gera bounding boxes. O segundo cria crops derivados. O terceiro roda um classificador nas regiões recortadas. O quarto, quando for útil ao projeto, converte caixas em máscaras via um modelo de segmentação apropriado — seja um segmentador treinado no próprio domínio, seja um modelo promptável como SAM 2. O próprio Ultralytics já documenta object cropping a partir de detecções, fornece modelos de classificação treináveis e oferece modelos de segmentação; a Meta documenta o SAM 2 como modelo unificado de segmentação em imagem e vídeo com prompts de caixa. ²⁴

Em termos de produto, isso pede um **editor de pipeline** em vez de uma simples tela de opções. O usuário deve conseguir montar algo como:

“modelo de detecção A” → “filtro por confiança e classe” → “gerador de crops com padding” → “modelo de classificação B” → “segmentação por caixa com modelo C” → “revisor humano” → “publicar dataset derivado”.

Essa abordagem é mais flexível e mais adequada ao que vocês querem como automação progressiva. A capacidade técnica para esse tipo de orquestração existe: Ultralytics provê os modos e modelos; CVAT provê entrada e saída amigáveis de anotação; MLflow registra cada execução. ²⁵

Para o caso específico de **gerar dataset de classificação a partir da detecção**, eu transformaria isso em uma funcionalidade formal chamada “Dataset derivado”. A regra seria: cada crop precisa guardar referência para imagem de origem, anotação de origem, track de origem quando houver vídeo, padding aplicado, política de borda, classe usada, modelo que gerou a detecção, score da detecção e eventuais correções humanas posteriores. Isso é coerente com a ideia de object cropping e com a necessidade de rastrear linhagem de dados ao longo do tempo. ²⁶

O **padding** na classificação deve existir, mas como política de geração, não como gambiarra visual. Em alguns domínios a classe depende do contexto imediato do objeto; em outros, contexto demais atrapalha. Minha sugestão é oferecer presets como “apertado”, “equilibrado” e “contextual”, com preview de alguns exemplos antes da geração do dataset derivado. Como a decisão do padding é de produto e domínio, não existe um valor oficial universal; o importante é torná-lo configurável e rastreável. A parte factual que sustenta essa decisão é que o object cropping existe, e modelos de classificação podem ser treinados sobre esses recortes. ²⁷

Para vídeo, a regra é ainda mais importante: **não gerar crops de todos os frames**. O Ultralytics suporta tracking multiobjeto com IDs persistentes, inclusive em detect e segment, e isso deveria ser usado para amostrar crops de forma inteligente por track, evitando milhares de imagens quase idênticas. Em vez de gerar um crop por frame, a política recomendada é algo como “no máximo N crops por track”, “respeitar intervalo mínimo entre frames”, “preferir keyframes e frames com mudança de escala/pose”,

e “manter todos os crops do mesmo track no mesmo split”. Isso reduz redundância e evita vazamento entre treino e validação. ²⁸

Também há uma regra operacional crítica para a integração com o CVAT: **evitar o fluxo destrutivo de exportar/importar anotações sempre que possível**. A documentação do CVAT deixa claro que, ao subir anotações, as anotações existentes podem ser removidas, e as APIs de import usam `replace` como padrão se não houver outro modo definido. O caminho mais seguro é usar a Auto-annotation API do CVAT, os endpoints de atualização incremental e jobs dedicados de revisão, em vez de ficar importando arquivos completos sobre tarefas já em andamento. ²⁹

Por fim, para novos lotes de dados, a regra precisa respeitar o próprio modelo do CVAT: **depois que a task é criada, não se adiciona mais mídia nela; cria-se outra task**. Isso é relevante para a sua noção de releases e de evolução do dataset. Cada novo lote deve entrar como entidade nova ou como task irmã no mesmo projeto, em vez de tentar “mutar” a task anterior indefinidamente. ³⁰

Regras para rastreabilidade, releases, qualidade e auditoria

A ferramenta deve assumir uma posição firme: **dados, decisões humanas e modelos precisam ser entidades versionadas e auditáveis**. O MLflow Model Registry descreve a ligação entre uma versão de modelo e a run que a produziu, com versionamento, comparação e rollback; o MLflow Dataset Tracking descreve a linhagem do dataset; e o CVAT oferece Ground Truth job, review mode e quality analytics para controlar a qualidade da anotação. ³¹

Por isso, eu recomendo que a especificação funcional inclua pelo menos estas entidades persistentes: `DatasetRelease`, `TrainingRun`, `ModelVersion`, `PipelineRun`, `DerivedAsset`, `ReviewDecision`, `AnnotationRevision` e `TrackRevision`. Cada uma existe para responder uma pergunta concreta: qual dataset treinou este modelo, quais decisões humanas corrigiram esta anotação, qual pipeline gerou este crop, qual bounding box virou essa máscara, e em que momento o modelo passou a ser considerado aprovado. Essa modelagem não vem pronta nos produtos citados, mas é a consequência mais natural do que MLflow e CVAT já oferecem como lineage, runs, registry, QA e jobs. ³²

Existe uma regra que eu considero obrigatória: **nenhum treino pode apontar para um dataset “vivo”**. Todo treino precisa apontar para uma release imutável do dataset, com lista fechada de imagens/frames, classes, split de validação e artefatos derivados. Isso é diretamente coerente com a lógica de dataset tracking e com a necessidade de reprodutibilidade que o MLflow destaca. ¹³

Outra regra importante é separar **qualidade da anotação** de **qualidade do modelo**. O CVAT já fornece validação de tarefas, review mode, feedback imediato e quality analytics. A sua interface deve consolidar isso em duas telas ou dois blocos: um para “o dado está bom?” e outro para “o modelo está bom?”. Misturar essas duas coisas impede decisões corretas, porque um mAP baixo pode significar problema no modelo, no dataset, no split, na diversidade do lote ou na qualidade da anotação. ³³

No plano de auditoria, toda decisão rápida do usuário deveria virar evento: aceitou, rejeitou, reclassificou, corrigiu geometria, aplicou ao track inteiro, desfez, escalou. E cada evento precisa guardar ator, timestamp, objeto afetado, motivo, origem da anotação e, quando houver, confiança do modelo que gerou a proposta. Isso é o que permitirá construir métricas realmente úteis, como “taxa de correção por classe”, “taxa de falso positivo por modelo”, “tempo médio de revisão por tipo de erro” e “ganho de mAP por 1.000 anotações adicionais”. O suporte factual para esse desenho está na combinação de experiment tracking do MLflow com a estrutura de jobs, eventos e QA do CVAT. ³⁴

Existe ainda um ponto jurídico/técnico que vale registrar no documento de regras: se a ferramenta virar um produto interno ou externo com Ultralytics incorporado no fluxo, a questão de licenciamento não deve ser ignorada. A própria documentação da Ultralytics destaca a existência de licença enterprise para integração em produtos e serviços de negócio fora das obrigações do AGPL. ³⁵

Arquitetura recomendada para a primeira versão e para a evolução futura

A arquitetura mais sólida para esse projeto é: **CVAT como camada de anotação, orquestrador próprio como camada de produto, workers de treino/inferência em contêineres, MLflow como repositório de runs, métricas e modelos, e armazenamento de objetos** para datasets, checkpoints, crops e artefatos. Essa composição encaixa com o que cada componente faz melhor e evita sobrecarga no frontend. ³⁶

Na interface, eu criaria sete áreas principais: **Dados, Anotar, Revisar, Treinar, Modelos, Pipelines e Auditoria**. “Anotar” e “Revisar” ficam muito próximos do fluxo do CVAT, com a sua camada de revisão rápida por teclado. “Treinar” é o painel de jobs, parâmetros e métricas. “Modelos” mostra histórico, comparação, registro e promoção. “Pipelines” traz o editor de encadeamento de detecção, classificação, segmentação e geração de datasets derivados. “Auditoria” mostra quem mudou o quê, quando e por qual motivo. Essa organização reflete bem as capacidades já existentes em CVAT, Ultralytics e MLflow. ³⁷

Para a primeira entrega, eu faria um recorte muito pragmático. Primeiro: revisão rápida com teclado, aceitação/rejeição/correção de classe e rastreabilidade mínima das decisões. Segundo: tela de treino com parâmetros essenciais e gráficos ao vivo. Terceiro: histórico de runs ligado a dataset release e validação fixa. Quarto: dataset derivado de classificação a partir de detecção com padding configurável e preview. Quinto: pipeline simples “detecção → crop → classificação”. Sexto: vídeo com interpolação do CVAT e revisão por keyframes. Tudo isso já gera muito ganho sem exigir que toda a ambição final seja implementada de uma vez. ³⁸

A resposta final, então, é objetiva: **sim, vale muito a pena encaixar métricas detalhadas, histórico de evolução, controle de parâmetros e pipelines na ferramenta**. O que muda é que a máquina de estados precisa evoluir para um modelo com estados de projeto, execução assíncrona e item anotado em paralelo. Feita essa mudança, o restante não só encaixa como passa a fazer sentido estruturalmente: treino em tempo real vira estado observável, pipeline vira estado executável, e evolução do modelo deixa de ser uma percepção difusa e vira um histórico mensurável, comparável e auditável. ³⁹

¹ ² ²⁵ ³⁶ Auto-annotation API | CVAT
https://docs.cvat.ai/docs/api_sdk/sdk/auto-annotation/

³ ⁴ ³⁹ Tasks | CVAT
<https://docs.cvat.ai/docs/workspace/tasks-page/>

⁵ Automated QA, Review & Honeypots | CVAT
<https://docs.cvat.ai/docs/qa-analytics/auto-qa/>

⁶ MLflow Integration for Ultralytics YOLO | Ultralytics
<https://docs.ultralytics.com/integrations/mlflow>

- 7 38 **Track mode | CVAT**
<https://docs.cvat.ai/docs/annotation/manual-annotation/modes/track-mode-basics/>
- 8 33 **Quality control | CVAT**
<https://docs.cvat.ai/docs/qa-analytics/quality-control/>
- 9 **engine.trainer API Reference | Ultralytics**
<https://docs.ultralytics.com/reference/engine/trainer>
- 10 22 **System Metrics | MLflow AI Platform**
<https://mlflow.org/docs/latest/ml/tracking/system-metrics/>
- 11 **ML Experiment Tracking | MLflow AI Platform**
<https://mlflow.org/docs/latest/ml/tracking/>
- 12 **Model Validation with Ultralytics YOLO | Ultralytics**
<https://docs.ultralytics.com/modes/val>
- 13 32 **ML Dataset Tracking | MLflow AI Platform**
<https://mlflow.org/docs/latest/ml/dataset/>
- 14 **Customizing Trainer | Ultralytics**
<https://docs.ultralytics.com/guides/custom-trainer>
- 15 **Tracking Hyperparameter Tuning with MLflow | MLflow AI Platform**
<https://mlflow.org/docs/latest/ml/getting-started/hyperparameter-tuning/>
- 16 17 **Model Training with Ultralytics YOLO | Ultralytics**
<https://docs.ultralytics.com/modes/train>
- 18 19 **Data Loading Optimization in PyTorch — PyTorch Tutorials 2.13.0+cu130 documentation**
https://docs.pytorch.org/tutorials/intermediate/intermediate_data_loading_tutorial.html
- 20 21 **Resource constraints | Docker Docs**
https://docs.docker.com/engine/containers/resource_constraints/
- 23 **Models Supported by Ultralytics**
https://docs.ultralytics.com/models?utm_source=chatgpt.com
- 24 26 27 **Extract and Crop Objects with YOLO26 | Ultralytics**
<https://docs.ultralytics.com/guides/object-cropping>
- 28 **YOLO Multi-Object Tracking in Video | Ultralytics**
<https://docs.ultralytics.com/modes/track>
- 29 30 **Import annotations and data to CVAT | CVAT**
https://docs.cvat.ai/docs/dataset_management/import-datasets/
- 31 **ML Model Registry | MLflow AI Platform**
<https://mlflow.org/docs/latest/ml/model-registry/>
- 34 **REST API**
<https://mlflow.org/docs/latest/rest-api.html>
- 35 **YOLO Object Detection & Segmentation | Ultralytics**
https://docs.ultralytics.com/?utm_source=chatgpt.com
- 37 **Webhooks | CVAT**
<https://docs.cvat.ai/docs/administration/community/advanced/webhooks/>